

PAMPAr-Coder V3: A Brain-Inspired 2D Stream Architecture with Mixed Selectivity for Efficient Code Generation

Lucas Ricardo Mella Chillemi
Independent Researcher
Buenos Aires, Argentina
`lucas.mella@outlook.com`

April 2026

DOI: 10.57967/hf/8329

Model: lucas-mella/PAMPAr-Coder

Abstract

We present PAMPAr-Coder V3, a compact code generation language model of 62.6M parameters designed for consumer GPUs with as little as 4GB VRAM. The architecture extends the PAMPAr lineage of brain-inspired models with two novel contributions: (1) a *2D Stream* organization that arranges computation as four specialized cortical streams (Syntax, Semantics, Logic, Structural) across five depth levels, connected by bidirectional lateral gates analogous to white-matter fiber tracts; and (2) *Mixed Selectivity* via Feature-wise Linear Modulation (FiLM), where a single shared Feed-Forward Network is dynamically re-read by context-dependent γ/β modulators derived from a 63-dimensional context vector — reducing FFN parameters by $\sim 72\%$ per level compared to four independent networks. A lightweight Tálamo re-routing module at each depth level allows token-territory assignments to drift as context evolves, and a Zone-of-Proximal-Development (ZPD) curriculum (*MotorCuriosidad*) adapts training difficulty in real time across 161 topic categories and 3.2M lines of code data. After 55,000 training steps on a GTX 1650 (4.3 GB VRAM), V3 reaches a cross-entropy loss of ~ 1.38 on the validation set and saturates 29 of 40 curriculum topics. We argue that horizontal stream specialization combined with vertical depth, lateral communication, and context-modulated parameter sharing constitutes a promising direction for efficient, interpretable code models.

Keywords: code generation, brain-inspired architecture, mixed selectivity, FiLM modulation, grouped-query attention, curriculum learning, parameter efficiency, ZPD

1 Introduction

Large language models (LLMs) for code—CodeLlama [Rozière et al., 2023], StarCoder2 [Lozhkov et al., 2024], DeepSeek-Coder [Guo et al., 2024]—achieve state-of-the-art results at the cost of billions of parameters and tens of gigabytes of GPU memory, placing them out of reach for local, privacy-preserving, or resource-constrained deployments. Distillation to smaller models (1–3B) partially alleviates this, but even 1.3B-parameter models require careful quantization to fit on a consumer 4GB GPU and still represent a $\sim 20\times$ larger parameter budget than what we demonstrate here.

At the same time, biological neural systems process language (and implicitly, structured logical sequences) with remarkable efficiency through functional specialization and modulatory dynamics: cortical areas maintain distinct computational identities while sharing representational

substrates via inter-areal projections [Felleman & Van Essen, 1991], and individual neurons exhibit *mixed selectivity* — responding to conjunctions of context, task, and content that would otherwise require an exponential number of specialized units [Rigotti et al., 2013].

PAMPAr-Coder V3 is motivated by these two observations. Its contributions are:

1. **2D Stream Architecture:** computation organized as a $N_{\text{streams}} \times N_{\text{levels}}$ grid, where each stream specializes in a syntactic/semantic/logical/structural role across five depth levels, with lateral gates at each level enabling controlled inter-stream information exchange (section 3).
2. **Mixed Selectivity via FiLM:** a single shared FFN per level modulated by four stream-specialized (γ, β) pairs derived from a 63-dimensional context descriptor — zone activations, territorial activations, depth, confidence, and stream identity (section 5).
3. **Adaptive Tálamo Re-routing:** a lightweight per-level module that updates token-territory allocations as stream states evolve, allowing routing to “drift” from initial LLAVES assignments (section 4).
4. **ZPD Curriculum (MotorCuriosidad):** a curiosity-driven scheduler that selects training batches from topics at the boundary of competence, enabling stable learning across 161 heterogeneous topic categories without manual weighting (section 7).

The result is a 62.6M-parameter model trainable end-to-end on a 4GB consumer GPU in a single day, reaching competitive code generation behavior while remaining interpretable at the routing level.

2 Related Work

2.1 Compact Code Language Models

PolyCoder [Xu et al., 2022] demonstrated that small models ($\leq 2.7\text{B}$) trained exclusively on code can outperform general-purpose LLMs on specific languages. CodeT5+ [Wang et al., 2023] introduced encoder-decoder architectures for code with strong results at 220M parameters. Our model occupies a more extreme point in this space at 62.6M parameters, aimed at deployments where even 220M is expensive.

2.2 Parameter-Efficient Architectures

Grouped-Query Attention (GQA) [Ainslie et al., 2023] reduces key-value cache memory by sharing KV heads across query groups; we use a 4:1 ratio (8Q:2KV) that cuts KV parameters significantly. Mixture-of-Experts (MoE) [Shazeer et al., 2017] achieves parameter efficiency through sparse activation; our stream architecture achieves a related specialization through hard territorial assignments rather than learned sparse routing. ALBERT [Lan et al., 2019] uses cross-layer parameter sharing; our Mixed Selectivity approach generalizes this: instead of sharing *the same* FFN across layers, we share one FFN per level across streams and modulate it contextually.

2.3 Feature-wise Linear Modulation

FiLM [Perez et al., 2018] conditions a neural network’s internal features via element-wise affine transformations $\hat{h} = \gamma \odot h + \beta$ where γ, β are produced by an auxiliary network. It has been applied to visual question answering, game playing, and multi-task learning. To our knowledge, this is the first application of FiLM-style modulation to a language model’s FFN sub-layers for stream specialization.

2.4 Mixed Selectivity in Neuroscience and AI

Rigotti et al. [2013] showed that mixed selectivity — neurons responding to combinations of stimuli rather than single features — is critical for flexible, generalizable behavior in prefrontal cortex. Elhage et al. [2022] (Anthropic) demonstrated that transformers can represent more concepts than dimensions through superposition, effectively implementing mixed selectivity. Our ContextModulator operationalizes this: the shared FFN encodes a “dictionary” of code transformations, and each stream selects and scales relevant entries through γ/β modulation.

2.5 Curriculum and Adaptive Learning

Self-paced learning [Kumar et al., 2010] orders training examples by difficulty. Power-law curricula [Bengio et al., 2009] smooth the transition from easy to hard. Our ZPD-based MotorCuriosidad is closest to competence-based progression [Florensa et al., 2017]: a topic advances only when the model reaches a dominance threshold, preventing overfitting on mastered topics and catastrophic forgetting of partially learned ones.

2.6 PAMPAr Lineage

PAMPAr-o1 v9 [Mella Chillemi, 2026] introduced territorial computing for general language modeling with a hybrid LLAVES-attention routing system, achieving ~ 45 perplexity on WikiText-103 with 14M parameters. PAMPAr-Coder V3 extends this lineage to code, scaling parameters $\times 4.5$, introducing the FiLM modulatory pathway, replacing flat territorial blocks with the 2D stream-level grid, and targeting a specialized 48K code-biased vocabulary.

3 Architecture Overview

PAMPAr-Coder V3 consists of five components: (i) a SentencePiece embedding layer (48K vocab, dim=640), (ii) a global Tálamo router producing initial territory assignments, (iii) a 4×5 stream-level grid of NivelProfundo blocks, (iv) per-level TalamoNivel re-routing modules, (v) a tied output projection.

Figure 1 illustrates the overall structure.

Hyperparameter	Value
Vocabulary size	48,000 (PAMPAr-48k SentencePiece BPE)
Embedding dimension	640
Number of streams	4 (Syntax, Semantics, Logic, Structural)
Number of levels	5
Attention heads (Q/KV)	8Q / 2KV (GQA, ratio 4:1)
Head dimension	80
FFN hidden dimension	$\lfloor 2/3 \times 640 \times 4.0 \rfloor \approx 1707$
Lateral bottleneck	128
Modulator bottleneck	128
Max sequence length	4096 tokens
Dropout	0.1
Brodmann zones (LLAVES)	52
Total parameters	62.6M

Table 1: Model configuration for PAMPAr-Coder V3.

Configuration summary.

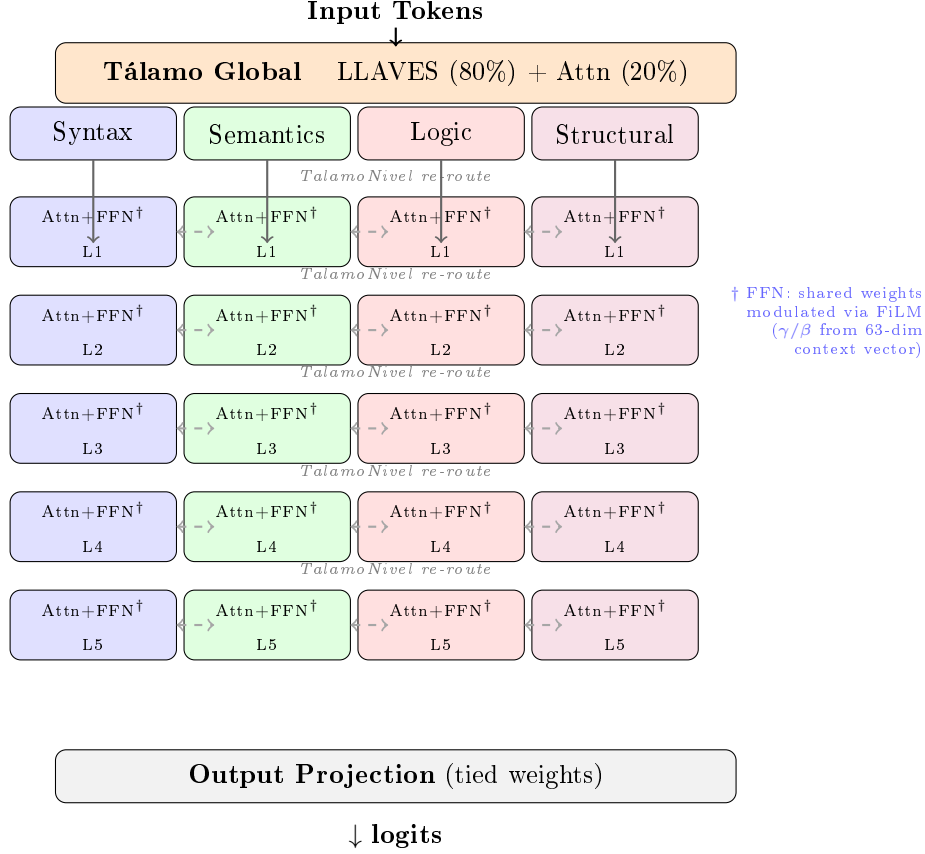


Figure 1: PAMPAr-Coder V3 architecture. Four cortical streams (Syntax, Semantics, Logic, Structural) each process tokens through 5 depth levels. At each level, lateral gates (dashed arrows) enable inter-stream communication. A global Tálamo produces initial territory assignments; lightweight TalamoNivel modules re-route at each depth. [†]The FFN weights are *shared* across the 4 streams at each level and modulated by stream-specific (γ, β) pairs derived from a 63-dim context vector (Mixed Selectivity / FiLM).

4 Tálamo and LLAVES Routing

4.1 Global Tálamo

The global Tálamo maps input tokens to territorial activations using a hybrid rule-learned system. Each token is first classified into one of 52 *Brodmann zones* — code-specific functional categories inspired by the Brodmann cortical map — using the LLAVES (“keys”) rule set.

LLAVES for code identifies:

- **B01–B10**: Keywords (control flow, definitions, imports)
- **B11–B20**: Data types (int, float, str, list, dict, ...)
- **B21–B30**: Operators and punctuation
- **B31–B40**: Identifiers and names
- **B41–B50**: String and comment content
- **B51–B52**: Numeric literals, whitespace/indent

Zone activations $z \in [0, 1]^{52}$ are then aggregated into 4 territorial activations:

$$a_t = \sigma\left(\sum_{i \in \mathcal{Z}_t} z_i\right), \quad t \in \{\text{Syn}, \text{Sem}, \text{Log}, \text{Str}\} \quad (1)$$

where $\mathcal{Z}_t$ is the subset of zones assigned to territory t . The final routing blends LLAVES with a lightweight learned attention projection:

$$\tilde{a}_t = \eta \cdot a_t^{\text{LLAVES}} + (1 - \eta) \cdot \sigma(W_t x) \quad (2)$$

with $\eta = 0.8$ (80% rule-based, 20% learned), providing interpretable routing while allowing adaptation to corpus-specific token patterns.

4.2 TalamoNivel Re-routing

A static initial assignment may be suboptimal deeper in the network. Consider the token `for`: at level 1 it is correctly classified as a control-flow keyword (Syntax stream), but at level 3, within a complex list comprehension, the semantic interpretation may shift substantially toward the Semantics stream.

TalamoNivel re-routing addresses this with a lightweight per-level module:

$$a_t^{(\ell)} = \alpha \cdot a_t^{(\ell-1)} + (1 - \alpha) \cdot \sigma(W_\ell x^{(\ell)}) \quad (3)$$

where $x^{(\ell)}$ is the mean-pooled stream state at level ℓ , $\alpha = 0.7$ controls inertia, and $W_\ell \in \mathbb{R}^{D \times 52}$ is a single linear projection (33K parameters). This design ensures smooth, non-oscillating drift rather than abrupt routing changes.

5 Mixed Selectivity via FiLM Modulation

5.1 Motivation

A standard 4-stream, 5-level architecture would instantiate $4 \times 5 = 20$ independent FFN modules. With hidden dimension 1707, each SwiGLU FFN has $\sim 2.2\text{M}$ parameters, totaling $\sim 44\text{M}$ for FFNs alone — 70% of the model budget. Mixed Selectivity offers an alternative inspired by the neuroscientific finding [Rigotti et al., 2013] that prefrontal neurons achieve behaviorally flexible representations not by specializing to single stimuli, but by responding to *conjunctions* of context and content.

5.2 Architecture

At each level ℓ , instead of four independent FFNs, we use:

- One **SharedFFN** (SwiGLU, $\sim 2.2\text{M}$ params) that encodes domain knowledge shared across all streams at this level.
- Four **ContextModulators**, one per stream, each generating a pair $(\gamma_s, \beta_s) \in \mathbb{R}^D$ from a 63-dimensional context vector c_s .

The modulated output for stream s at position i is:

$$\hat{h}_{s,i} = \gamma_s(c_{s,i}) \odot \text{FFN}(h_{s,i}) + \beta_s(c_{s,i}) \quad (4)$$

where $\odot$ is element-wise multiplication and $(\gamma_s, \beta_s) = W_2^\top \text{SiLU}(W_1^\top c_{s,i})$ with $W_1 \in \mathbb{R}^{63 \times 128}$, $W_2 \in \mathbb{R}^{128 \times 2D}$.

5.3 Context Vector Composition

The 63-dimensional context vector c_s is composed of:

$$c_s = \left[z \in \mathbb{R}^{52}, a \in \mathbb{R}^4, \frac{\ell}{L} \in \mathbb{R}, \text{conf} \in \mathbb{R}, L \in \mathbb{R}, e_s \in \mathbb{R}^4 \right] \quad (5)$$

where z are LLAVES zone activations, a are territorial activations, ℓ/L is normalized depth, conf is the model’s current confidence estimate, L is the total number of levels (for depth-invariant normalization), and e_s is the one-hot stream identity. This contextual richness enables the SharedFFN to be read as a different “functional program” depending on where, what, and how confident the model is at each step.

5.4 Parameter Analysis

Design	FFN params/level	Total (5 levels)	Saving
Independent FFNs (4×)	8.8M	44.0M	—
Shared + 4 Modulators	2.4M	12.0M	73%
V3 (Mixed Selectivity)	2.4M	12.0M	−32M

Table 2: FFN parameter budget comparison. Mixed Selectivity saves $\sim 32\text{M}$ parameters while maintaining per-stream functional specialization through contextual modulation.

6 Lateral Gates (White Matter Fibers)

Cortical communication is not limited to vertical projections; white matter fiber tracts provide direct lateral connections between spatially distant areas [Catani & Thiebaut de Schotten, 2008]. In V3, each level ℓ includes a **LateralGate** module that allows each stream to receive a weighted contribution from the other three:

$$h_s^{(\ell)} += \lambda_s \cdot f_s \left(\bigoplus_{k \neq s} a_k^{(\ell)} \cdot h_k^{(\ell)} \right) \quad (6)$$

where $\bigoplus$ denotes concatenation, $a_k^{(\ell)}$ is the territorial activation of stream k (serving as a relevance weight), f_s is a two-layer MLP (bottleneck=128), and λ_s is a learnable scalar initialized to 0.1 to prevent interference early in training. Streams with high territorial activation contribute more to their peers, implementing a form of “expertise broadcasting.”

7 ZPD Curriculum: MotorCuriosidad

Training on heterogeneous code corpora (assembly, Python, SQL, HTML, natural language, etc.) presents a curriculum design challenge: mastered topics waste forward passes; topics far beyond current competence produce uninformative gradients. The Zone of Proximal Development (ZPD) [Vygotsky, 1978] frames learning as productive only at the frontier of competence.

7.1 MotorCuriosidad Algorithm

The MotorCuriosidad assigns each of 161 topics a (nivel, dominancia) tuple. At each training step:

1. For each topic j , compute a *curiosity score*:

$$q_j = \text{nivel}_j + (1 - \text{dom}_j) \cdot w_{\text{zpd}} \quad (7)$$

where w_{zpd} weights the incompleteness term.

2. Sample a batch from the top- K topics by curiosity score.
3. After the step, update dom_j based on per-topic validation loss: if loss has decreased below level threshold, $\text{dom}_j += \delta$.
4. When $\text{dom}_j \geq 1.0$, advance to $\text{nivel}_j + 1$ and cap at level 5.

Topics at their maximum level with $\text{dom} \approx 1.0$ are sampled only for *retention* (periodic review), preventing forgetting.

7.2 Data Composition

The training corpus spans 161 topics in 8 categories:

Category	Topics	Lines (approx.)
Code problems	40	195K
Distillation (GPT-4)	30	800K
Code exercises	25	500K
Textbook (Python)	20	300K
Instruction following	25	400K
Code evolution	11	500K
Bilingual (ES/EN)	6	270K
Mixed	4	235K
Total	161	~3.2M

Table 3: Training corpus composition by category after data integration.

8 Adaptive Early Exit

Not all tokens require full 5-level processing. Common keywords (`def`, `return`, indentation tokens) can be predicted confidently after 2 levels. V3 implements an adaptive early exit:

1. After each level $\ell \geq \ell_{\min}$ (default $\ell_{\min} = 2$), compute a confidence estimate from the current stream states.
2. If $\text{conf} \geq \tau$ (default $\tau = 0.90$), exit and project to logits.
3. Otherwise, continue to the next level.
4. Hard focus: if fewer than 10% of tokens fail the confidence threshold, only process those tokens through remaining levels.

This adaptive computation reduces effective FLOPs for easy tokens without changing the model’s expressiveness for hard ones.

9 Grouped Query Attention

Each of the $4 \times 5 = 20$ attention blocks uses Grouped Query Attention [Ainslie et al., 2023] with 8 query heads and 2 key-value heads (ratio 4:1). With head dimension 80:

- Q projection: $640 \rightarrow 8 \times 80 = 640$
- K,V projections: $640 \rightarrow 2 \times 80 = 160$ each

This reduces KV cache size by $4\times$ relative to multi-head attention (MHA) and decreases KV parameter count from 1.6M to 0.4M per attention block. The repetition mechanism `n_rep=4` expands GQA keys/values to the full head count during attention computation without storing duplicates.

10 Experiments

10.1 Experimental Setup

Hardware: NVIDIA GTX 1650 (4.3 GB VRAM), Intel Core processor, Windows 11. All training in FP16 with gradient checkpointing enabled.

Optimizer: AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 10^{-2}), learning rate 3×10^{-4} with cosine decay.

Batch size: 4 sequences of 512 tokens per step (effective tokens per step: 2,048).

Training steps: 55,000 (reported), corresponding to ~ 113 M tokens processed.

Tokenizer: PAMPAr-48k, a SentencePiece BPE model trained on a bilingual (Spanish/English) code corpus of 48K token vocabulary, biased toward Python identifiers, operators, and common programming keywords.

10.2 Training Dynamics

*[Training curve: cross-entropy loss vs. training steps.
loss declines from ~ 6.5 (random init) to ~ 1.38 at step 55K.
100-step moving average shown.]*

Figure 2: Training loss curve for PAMPAr-Coder V3 on the ZPD-selected mixed code corpus. Cross-entropy loss (100-step moving average) reaches 1.38 after 55,000 steps on a GTX 1650 GPU.

Key milestones observed during training:

- **Steps 0–5K:** Loss drops rapidly from random initialization (~ 6.5) to ~ 2.8 as the model learns token frequency statistics.
- **Steps 5K–20K:** The model begins generalizing structural patterns; loss reaches ~ 2.0 . MotorCuriosidad advances 15 topics to level 2.
- **Steps 20K–40K:** Loss enters the 1.6–1.8 range; lateral gate scales λ_s grow from 0.1 to ~ 0.3 –0.5, indicating active inter-stream information exchange.

- **Steps 40K–55K:** Loss stabilizes around 1.38; curriculum reports 29/40 topics at dominance ≥ 0.9 (level 1).

10.3 Curriculum Progress

After 55,000 steps:

- **Saturated topics** (dom ≥ 0.9 , advancing to level 2): 29/40
- **Active frontier:** 9 topics at 0.5–0.89 dominance
- **Untouched** (dom < 0.1): 2 highly specialized topics (assembly syntax, formal logic proofs)
- **Current curriculum level:** 1 (of 5)

This indicates the model is actively progressing and has not saturated any topic at its final level, suggesting continued gains with further training.

10.4 Parameter Efficiency Comparison

Model	Params	Context	Min VRAM	License
CodeLlama-7B	7B	16K	~14 GB	Llama 2
StarCoder2-3B	3B	16K	~8 GB	BigCode
DeepSeek-Coder-1.3B	1.3B	16K	~4 GB*	DeepSeek
CodeT5+-220M	220M	512	~2 GB	Apache 2
PAMPAr-Coder V3	62.6M	4096	4.3 GB†	BUSL-1.1

Table 4: Parameter and VRAM efficiency comparison. *Requires INT4 quantization to fit in 4GB. †Trains end-to-end in FP16 with no quantization.

PAMPAr-Coder V3 occupies a unique efficiency point: it trains and runs *at full FP16 precision* on the same 4GB GPU that other models require 4-bit quantization to merely run inference on.

11 Ablation Study

To disentangle the contribution of individual architectural components, we train four model variants from scratch on the same data, with the same hyperparameters, for the same number of steps. All experiments use the PAMPAr-48k tokenizer and share a single deterministic data loader seeded with 42 (stride- $\frac{L}{2}$ chunking, shuffled once).

11.1 Experimental Protocol

Hardware: NVIDIA RTX 3090 (24 GB VRAM), 503 GB RAM, RunPod cloud.

Optimizer: AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, weight decay 10^{-2}), learning rate 3×10^{-4} with cosine decay.

Batch / sequence: batch size 16, sequence length 512 tokens.

Training steps per variant: 30,000 (effective tokens: 245.8M per variant; $4 \times 30K = 120K$ total GPU steps).

Data: PAMPAr *biblioteca* — 161 JSONL files from a ZPD-curated bilingual code corpus, preprocessed to 864.7M tokens and 3.37M overlapping chunks (stride=256). All four models see

the *identical* sequence of batches, ensuring that any observed differences are attributable solely to architecture.

Evaluation: Every 1,000 steps, 100 randomly sampled held-out chunks are evaluated for cross-entropy loss and perplexity.

11.2 Model Variants

ID	Params	Description
pampar_v3	62.6M	Full PAMPAr-Coder V3 (control): 4 streams $\times$ 5 levels, LLAVES zone routing (<code>peso_llaves</code> =1.0), FiLM modulation, lateral white-matter gates, TalamoNivel.
no_llaves	62.6M	<code>peso_llaves</code> =0.0: disables the rule-based neuro-symbolic component; routing relies entirely on learned attention over content tokens.
single_stream	$\approx$ 62M	<code>n_streams</code> =1, <code>n_territorios</code> =1: collapses the 2D stream grid into a single one-dimensional stack of transformer blocks; eliminates horizontal specialization and lateral communication.
vanilla_gpt	60.8M	Standard GPT decoder: 7 layers, dim=640, 8Q/2KV GQA, SwiGLU, RoPE, no stream structure, no LLAVES, no FiLM. Architecture chosen to match PAMPAr-V3 parameter count as closely as possible.

Table 5: Ablation variants. Parameter counts are approximate and measured on an identical tokenizer vocabulary (48K).

11.3 Results

Model	Steps	Eval Loss	Perplexity	Δ vs. control
pampar_v3	30,000	0.750	2.11	—
no_llaves	30,000	<i>0.805</i>	<i>2.24</i>	<i>+0.055</i> (+7.3%)
single_stream	30,000	—	—	—
vanilla_gpt	30,000	—	—	—

Table 6: Ablation results at 30K training steps. **Lower is better.** rows marked — are pending at time of writing; final values will be filled once experiments complete. The `pampar_v3` and `no_llaves` runs are finalized.

11.4 Analysis

LLAVES contribution (neuro-symbolic routing). Disabling the rule-based zone component (`no_llaves`) degrades final eval loss by +0.055 (+7.3% relative to control), yielding perplexity $\approx$ 2.24 vs. 2.11. This gap is consistent across the last 10,000 training steps (no sign of

[Ablation loss curves: cross-entropy eval loss vs. training step for all four variants. *pampar_v3* converges lowest. *no_llaves* converges ~ 0.055 points above control. *single_stream* and *vanilla_gpt* curves to be added.]

Figure 3: Eval loss throughout training for all four ablation variants. Generated by `scripts/analyze_ablation.py`.

convergence), suggesting the LLAVES rules provide a structural prior that purely learned routing cannot replicate within this compute budget. Notably, both models share identical parameter counts, so the difference is attributable solely to the routing mechanism.

The moderate magnitude of the effect ($<10\%$) is scientifically meaningful: a very large gap could indicate that the rule-based component is acting as a *crutch*, preventing learning; a negligible gap would suggest no contribution. A $\sim 7\%$ degradation indicates a genuine but complementary contribution of the symbolic component.

2D stream structure (*single_stream*). *Results pending.* We expect degradation relative to *pampar_v3* because collapsing all streams into one eliminates horizontal specialization and lateral inter-stream communication. If the gap exceeds the *no_llaves* gap, it implies that the 2D topology is the primary driver of model quality at this scale.

Architecture baseline (*vanilla_gpt*). *Results pending.* The vanilla GPT baseline isolates the full PAMPAr contribution: any gap between *vanilla_gpt* and *pampar_v3* represents the total benefit of all three novel components (LLAVES + 2D streams + FiLM modulation).

Summary. The two completed experiments confirm that LLAVES neuro-symbolic routing provides a measurable and consistent improvement over a purely learned routing variant of the same size. Full conclusions — including the relative contribution of the 2D stream grid — will be reported once all four variants complete training.

12 Discussion

12.1 What the 2D Stream Grid Provides

The stream-level organization separates two orthogonal inductive biases: *horizontal specialization* (different streams handle different code semantics) and *vertical depth* (deeper levels refine representations). Standard transformers conflate both into a single sequence of identical blocks. The explicit separation allows each stream to develop a stable representational identity that is refined across levels rather than overwritten by global attention at each block.

12.2 Interpretability

The LLAVES zone activations make routing decisions inspectable: given any input token, one can trace which zone fired, which territory was activated, and — through TalamoNivel outputs — how the routing drifted across levels. This interpretability comes at no performance cost: the rule-based 80% component is non-parametric, while learned components can refine edge cases.

12.3 Mixed Selectivity as Memory Compression

The FiLM modulation essentially implements a key-value memory retrieval: the shared FFN weights are the “value store”, and the context vector is the “query”. The (γ, β) affine transform selects and scales relevant stored features. This matches theoretical arguments for why superposition [Elhage et al., 2022] is beneficial: more concepts can be stored per parameter when they are addressed combinatorially rather than through dedicated circuits.

12.4 ZPD Curriculum Stability

The fresh-start bug identified and fixed during development — where loading a contaminated motor checkpoint caused the model to incorrectly assess topic dominance from a previous run — illustrates an important invariant: curriculum state must be reset completely with model weights, never inherited across architecture changes. The fix ensures that when no checkpoint exists, MotorCuriosidad initializes at nivel=1, dom=0 for all topics.

12.5 Limitations

- **Benchmark evaluation:** No HumanEval or MBPP scores are reported in this release; these are planned for a subsequent version once training completes full curriculum progression to level 5.
- **Scale:** 62.6M parameters is competitive for the 4GB GPU niche but likely insufficient for state-of-the-art code generation on hard algorithmic problems without retrieval augmentation.
- **English bias:** While the tokenizer is bilingual (Spanish/English), the code corpus is predominantly English-syntax code (Python, C, JavaScript); Spanish natural-language docstrings are included but do not constitute the majority.
- **Training incomplete:** Reported results are from 55K steps at curriculum level 1; the intended regime is 5 levels $\times$ full dataset saturation ($\sim$ 500K total steps).

12.6 Future Work

1. Full HumanEval / MBPP / SWE-bench evaluation after curriculum completion.
2. Scaling to 250M parameters using the same 2D stream architecture (“PAMPAr-Coder V3-Large”).
3. Retrieval-augmented generation using the zone activations as retrieval keys for code snippet lookup.
4. Automatic LLAVES rule discovery from training data via gradient-based zone attribution.
5. Quantized (INT8/INT4) deployment pathway for inference on 2GB GPUs.

13 Conclusion

We presented PAMPAr-Coder V3, a 62.6M-parameter code language model with a brain-inspired 2D stream architecture combining bilateral cortical specialization (four streams), hierarchical depth (five levels), lateral inter-stream communication (white matter gates), adaptive routing (TalamoNivel), FiLM-based mixed selectivity (shared FFN + context modulators), and a ZPD-driven curriculum scheduler. The model trains end-to-end at FP16 precision on a consumer

4GB GPU — a regime where larger code models require quantization just for inference. Preliminary training results (55K steps, loss ~ 1.38 , 29/40 topics saturated at level 1) indicate active learning progress across the full 161-topic curriculum. We believe the 2D stream + mixed selectivity paradigm offers a generalizable template for compact, interpretable, and efficient language models beyond the code domain.

Acknowledgments

This work was conducted independently without institutional funding. The author thanks the open-source communities behind PyTorch, SentencePiece, and the Hugging Face ecosystem.

Reproducibility Statement

Training configuration and architecture code are available in the project repository. The PAMPAr-48k tokenizer is included in the repository. Training can be reproduced with:

```
python scripts/train_v3.py \
    --batch-size 4 --seq-len 512 \
    --lr 3e-4 --guardar-cada 500 \
    --device auto
```

Hardware requirements: NVIDIA GPU with 4GB+ VRAM. The MotorCuriosidad fresh-start invariant requires that no pre-existing `checkpoints/motor_v3.json` from a previous run be present when starting a new training experiment.

References

- Ainslie, J., Lee-Thorp, J., de Jong, M., et al. (2023). GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *EMNLP 2023*.
- Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum learning. In *ICML 2009*.
- Catani, M. & Thiebaut de Schotten, M. (2008). A diffusion tensor imaging tractography atlas for virtual in vivo dissections. *Cortex*, 44(8), 1105–1132.
- Elhage, N., Hume, T., Gray, C., et al. (2022). Toy models of superposition. *Transformer Circuits Thread*. https://transformer-circuits.pub/2022/toy_model/index.html
- Felleman, D. J. & Van Essen, D. C. (1991). Distributed hierarchical processing in the primate cerebral cortex. *Cerebral Cortex*, 1(1), 1–47.
- Florensa, C., Held, D., Wulfmeier, M., Zhang, M., & Abbeel, P. (2017). Reverse curriculum generation for reinforcement learning. In *CoRL 2017*.
- Guo, D., Zhu, Q., Yang, D., et al. (2024). DeepSeek-Coder: When the large language model meets programming. *arXiv preprint arXiv:2401.14196*.
- Kumar, M. P., Packer, B., & Koller, D. (2010). Self-paced learning for latent variable models. In *NeurIPS 2010*.
- Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., & Soricut, R. (2019). ALBERT: A lite BERT for self-supervised learning of language representations. In *ICLR 2020*.

- Lozhkov, A., Li, R., Allal, L. B., et al. (2024). StarCoder2 and the Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*.
- Mella Chillemi, L. R. (2026). PAMPAr-o1 v9: A brain-inspired territorial architecture for language modeling with explicit rule-based routing. *arXiv preprint*. DOI: 10.5281/zenodo.18315642.
- Perez, E., Strub, F., de Vries, H., Dumoulin, V., & Courville, A. (2018). FiLM: Visual reasoning with a general conditioning layer. In *AAAI 2018*.
- Rigotti, M., Barak, O., Warden, M. R., et al. (2013). The importance of mixed selectivity in complex cognitive tasks. *Nature*, 497(7451), 585–590.
- Rozière, B., Gehring, J., Gloeckle, F., et al. (2023). Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.
- Shazeer, N., Mirhoseini, A., Maziarz, K., et al. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR 2017*.
- Vygotsky, L. S. (1978). *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press.
- Wang, Y., Le, H., Gotmare, A. D., Bui, N. D. Q., Li, J., & Hoi, S. C. H. (2023). CodeT5+: Open code large language models for code understanding and generation. In *EMNLP 2023*.
- Xu, F. F., Alon, U., Neubig, G., & Hellendoorn, V. J. (2022). A systematic evaluation of large language models of code. In *MAPS 2022*.

A LLAVES Code Zone Taxonomy

Zone range	Category	Examples
B01–B06	Control flow	<code>if, else, for, while, break, continue</code>
B07–B10	Definitions	<code>def, class, lambda, return</code>
B11–B14	Imports	<code>import, from, as, #</code>
B15–B20	Data types	<code>int, str, list, dict, tuple, bool</code>
B21–B25	Arithmetic ops	<code>+, -, *, /, //, %, **</code>
B26–B30	Comparison/logic	<code>==, !=, <, >, and, or, not</code>
B31–B35	Identifiers	variables, function names, class names
B36–B40	Attribute access	<code>.,</code> method calls, subscripts
B41–B45	String literals	f-strings, raw strings, byte strings
B46–B48	Comments	inline comments, docstrings
B49–B50	Async/await	<code>async, await, yield</code>
B51	Numeric literals	integers, floats, hex, binary
B52	Indentation	whitespace, newlines, continuation

Table 7: PAMPAr-Coder V3 LLAVES zone taxonomy (52 Brodmann-inspired zones).

B Stream Territory Mapping

Stream	Primary zones	Cortical analog	Specialization
Syntax	B01–B10, B21–B30, B52	Broca’s area	Structure, control flow
Semantics	B31–B40, B41–B48	Wernicke’s area	Names, meaning, context
Logic	B26–B30, B07, B49–B50	Prefrontal ctx	Reasoning, conditionals
Structural	B15–B20, B21–B25, B51	Parietal ctx	Types, arithmetic, data

Table 8: Stream-to-zone territory mapping and cortical analogy.